

MACHINE LEARNING COURSE · CS-433

Underfitting and Overfitting

September 22, 2026

Martin Jaggi

credits to Mohammad Emtiyaz Khan & Rüdiger Urbanke

EPFL

Motivation

Models can be *too limited* or they can be *too rich*. In the first case we cannot find a function that is a good fit for the data in our model. We then say that we **underfit**.

In the second case we have such a rich model family that we do not just fit the underlying function but we in fact fit the noise in the data as well. We then talk about an **overfit**.

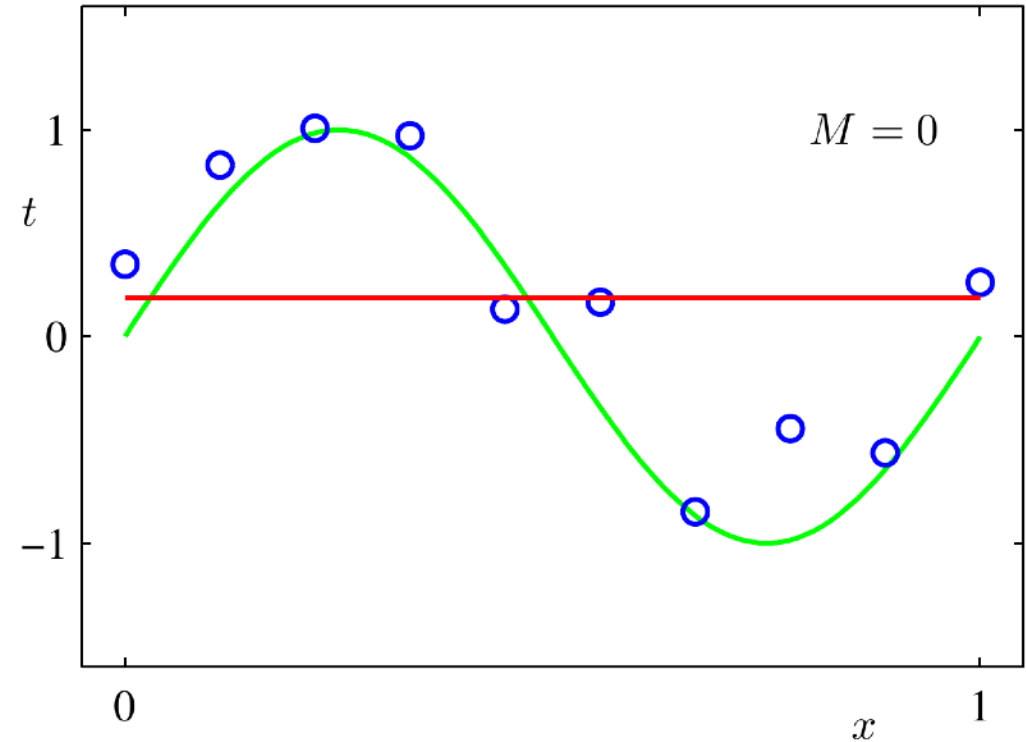
Both of these phenomena are undesirable. This discussion is made more difficult since all we have is data, and so we do not know a priori what part is the underlying signal and what part is noise.

Underfitting with linear models

It is easy to see that linear models might underfit. Consider a scalar case as shown in the figure.

The solid curve is the underlying function and the circles are the actual data. We assume there is a scalar function $g(x)$, but we do not observe $g(x_n)$ directly — only a noisy version of it:

$$y_n = g(x_n) + Z_n$$



Z_n is the noise, e.g. from measurement inaccuracies. The y_n are shown as blue circles.

Why a linear model cannot catch up

If our model family consists of only linear functions of the scalar input x , i.e.

$$\mathcal{H} = \{f_w(x) = w x\}$$

where w is a scalar constant (the slope of the function), then it is clear that we cannot match the given function accurately, regardless of how many samples we get and how small the noise is. We therefore will underfit.

Extended / augmented feature vectors

From the above example it might seem that linear models are too simple to ever overfit. But in fact, linear models are highly prone to overfitting, much more so than complicated models like neural nets.

Since linear models are inherently not very rich, the following is a standard “trick” to make them more powerful. To increase their representational power we typically “augment” the input. E.g. if the input (feature) is one-dimensional we might add a polynomial basis of arbitrary degree M ,

$$\phi(x_n) := [1, x_n, x_n^2, x_n^3, \dots, x_n^M]$$

Fitting the augmented model

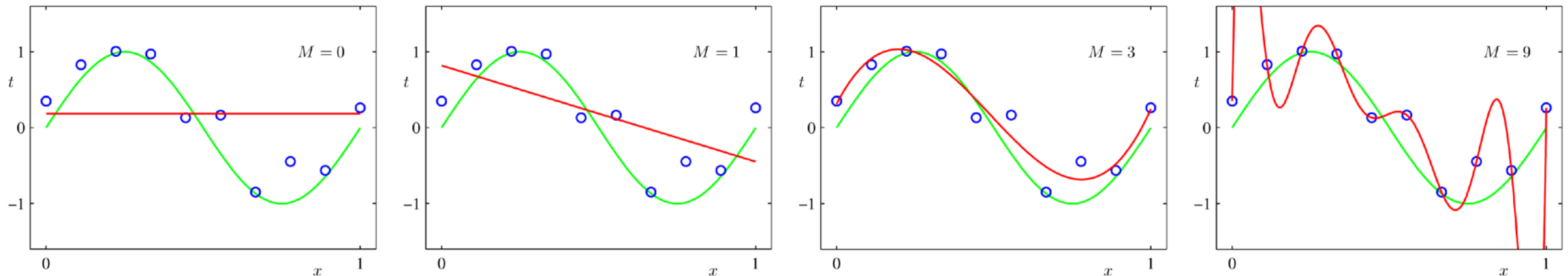
We then fit a linear model to this extended feature vector $\phi(x_n)$:

$$y_n \approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M =: \phi(x_n)^\top \mathbf{w}$$

The model is still linear in the weights — which is what lets us keep using least squares — but it is no longer linear in the input.

Overfitting with linear models

In the following four figures, circles are data points, the green line represents the “true function”, and the red line is the model. The parameter M is the maximum degree in the polynomial basis.



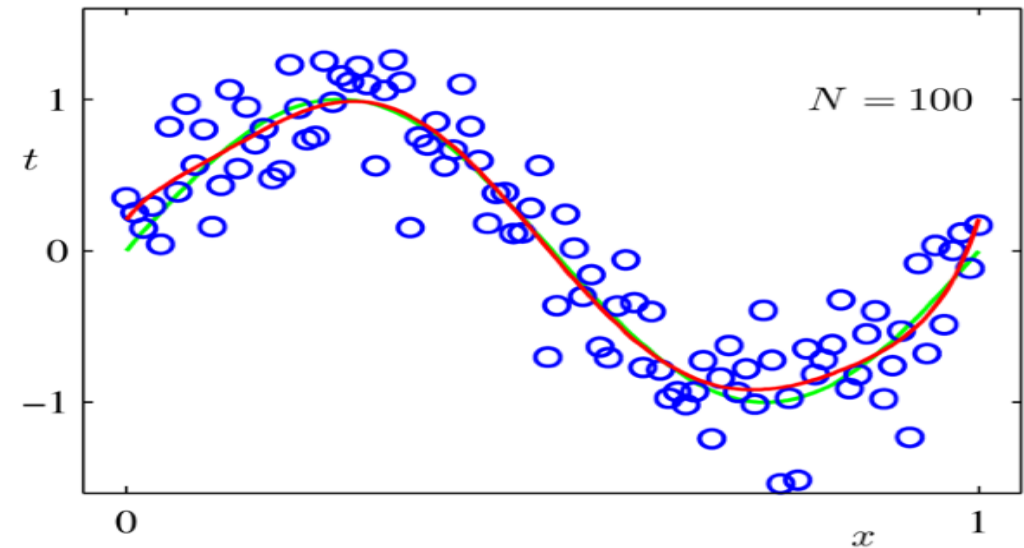
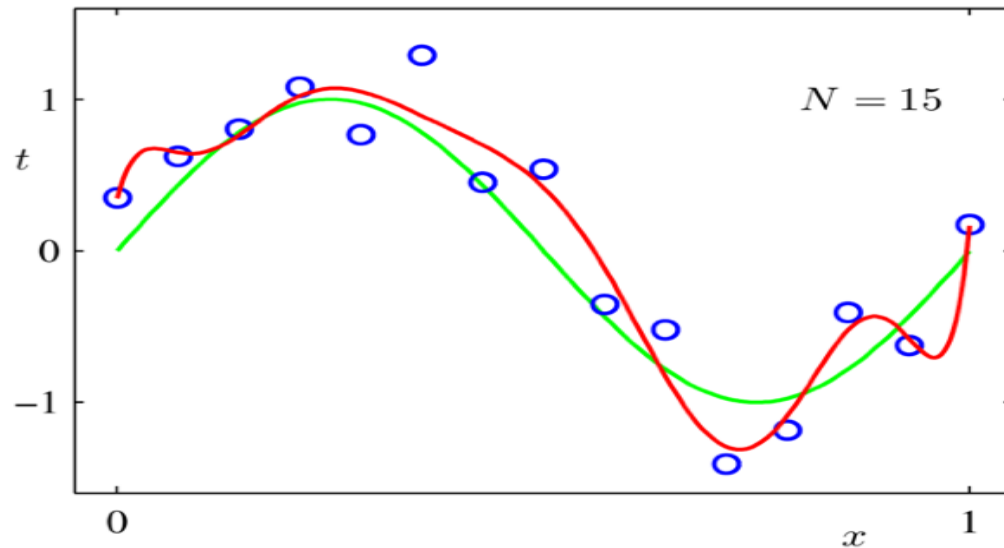
What the four fits show

For $M = 0$ (the model is a constant) the model is underfitting, and the same is true for $M = 1$. For $M = 3$ the model fits the data fairly well and is not yet so rich as to also fit the small “wiggles” caused by the noise.

But for $M = 9$ we now have such a rich model that it can fit every single data point, and we see severe overfitting taking place.

Does more data help?

What can we do to avoid overfitting? If you increase the amount of data (increase N , but keep M fixed), overfitting *might* reduce. Below, the same model complexity $M = 9$ with extra data.



A word about notation

If it is important to distinguish the original input x from the augmented input, then we will use $\phi(x)$ to denote this augmented input vector.

But we can consider this augmentation as part of the pre-processing, and then we might simply write x to denote the input. This will save us a lot of notation.